

Topic 4.4

4.4: Initializations, Assignments, Copy Constructors and =

If no constructor is provided for a class, the compiler provides one that takes no arguments and does nothing. This is the default constructor for the class. There are several instances when a copy must be made of a class object:

- When a class object is passed to a function by value
- When a function must return a class object as its result
- When one object is declared and initialized to an existing object

```
Distance abc( Distance d )
{
    Distance temp = d; // Copy constructor invoked
                       // because this is an
                       // initialization, not an
                       // assignment. The
                       // declaration:
                       // Distance temp(d);
                       // means exactly
                       // the same thing.

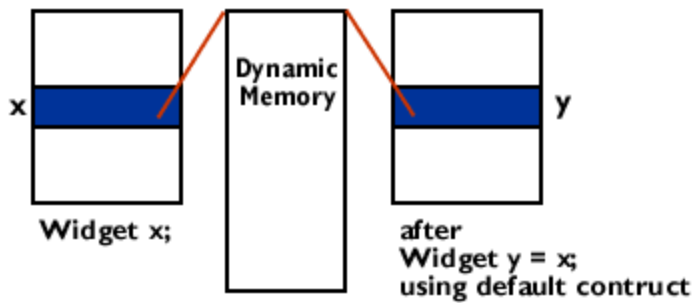
    ...
    return temp;      // Copy constructor invoked.
}

Distance a, b;
a = abc(b);           // Copy constructor invoked
                       // to pass b.
```

In such cases, the copy constructor is used. The copy constructor is a constructor with a single argument which is a reference to an object of the class:

```
class Distance
{
    ...
    Distance( Distance& d );
    ...
};
```

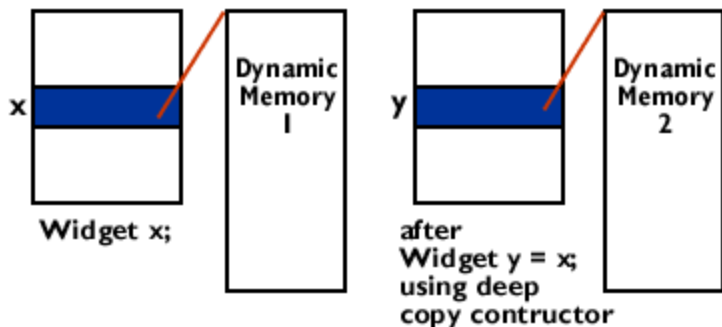
If a class does not define a copy constructor, a default is provided. The default copy constructor simply copies the data members from the original into the duplicate. This is often referred to a *shallow* copy. In simple cases such as the [Distance](#) class, this is OK. However, if our class includes dynamically allocated data, this is not sufficient. The result will be two objects pointing to the same dynamic memory, because only the pointer gets copied by the default copy constructor. Suppose class [Widget](#) includes dynamically allocated memory:



There are 2 major problems with this:

- Both `x` and `y` point at the same dynamic memory. If either changes it, both see the change. (This can actually be useful in reference-counting objects.)
- Since the dynamic memory is really associated with `x`, when `x` is destroyed its destructor will typically deallocate the memory. This leaves `y` pointing at space that is no longer allocated.

When a class involves dynamic memory, it is usually necessary to provide a copy constructor that replicates the dynamic memory. This is often called a *deep copy*.



When a statement such as

```
Distance a,b;
b = a; // This is a true assignment.
```

is executed, the overloaded operator `=` is invoked.

If no overloaded operator `=` is provided for the class, the default assignment operator is used, which simply performs a shallow copy from the source to the target of the assignment. In many cases, such as for the `Distance` class, this is OK.

Again, if the class involved dynamic memory, this may not be appropriate, in which case a class-specific version of operator `=` must be provided to perform a deep copy.

Note that the operator `=` and the copy constructor are subtly different. The copy constructor is *creating* the duplicate variable. The operator `=` is assigning to a value to an *existing* target variable.

There are some issues that must be dealt with by the operator `=`:

- What if the source and target are the same (for example, `a = a`)?
- Typically, the target's dynamic memory must be deallocated, then reallocated as required by the assignment.
- A reference to the target should be returned so assignments can be cascaded, for example, `a = b = c`;

Topic 4.5 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-4-dot-5>).